

姓名	李泽浩	学号	2025218895	实验成绩	
专业班级	软件工程 25-9 班	实验时间	4.19	实验地点	新安学堂

实验三：栈与队列综合实验

1 实验目的

- 1) 熟练掌握栈与队列的逻辑结构和运算特点；
- 2) 熟练掌握栈与队列的顺序存储结构和链式存储结构。
- 3) 熟练掌握栈与队列的有关算法设计，并在顺序结构和链式结构上实现。
- 4) 根据具体给定的需求，合理设计并实现相关结构和算法。

2 实验要求

- 1) 对用到的栈和队列进行合理封装；
- 2) 分别使用顺序存储和链式存储实现基本运算；
- 3) 设计算法实现相应的实验任务；
- 4) 程序运行、测试正确；
- 5) 实验程序有较好可读性，各运算和变量的命名直观易懂，符合软件工程要求；
- 6) 程序有适当的注释，程序的书写要采用缩进格式。

3 实验任务

编写算法求解下列问题：

- 1) 对一个合法的数学表达式来说，其中的各大小括号“{”，“}”，“[”，“]”，“（”和“）”应是相互匹配的。设计算法对以字符串形式读入的表达式 S，判断其中的各括号是否是匹配的。比如：“{[]O{}}”是匹配的，“{[()]} ”就是不匹配的。
- 2) 假设栈的输入序列为 1、2、3、...、n，设计算法求出所有可能的出栈序列。比如输入 1、2、3、4、5，可能出栈的序列为 12345、13452 等 42 个。
- 3) 约瑟夫环问题(Josephus loop problem)

据说著名犹太历史学家 Josephus 有过以下的故事：在罗马人占领乔塔帕特后，39 个犹太人与 Josephus 及他的朋友躲到一个洞中，39 个犹太人决定宁愿死也不要被敌人抓到，于是决定了一个自杀方式，41 个人排成一个圆圈，由第 1 个人开始报数，每报数到第 3 人该人就必须自杀，然后再由下一个重新报数，直到所有人都自杀身亡为止。然而 Josephus 和他的朋友并不想遵从。首先从一个人开始，越过 k-2 个人（因为第一个人已经被越过），并杀掉第 k 个人。接着，再越过 k-1 个人，并杀掉第 k 个人。这个过程沿着圆圈一直进行，直到最终只剩下一个人留下，这个人就可以继续活着。问题是，给定了和，一开始要站在什么地方才能避免被处决。Josephus 要他的朋友先假装遵从，他将朋友与自己安排在第 16 个与第 31 个位置，于是逃过了这场死亡游戏。

17 世纪的法国数学家加斯帕在《数目的游戏问题》中讲了这样一个故事：15 个教徒和 15 个非教徒在深海上遇险，必须将一半的人投入海中，其余的人才能幸免于难，于是想了一个办法：30 个人围成一圈，从第一个人开始依次报数，每数到第九个人就将他扔入大海，如此循环进行直到仅余 15 个人为止。问怎样排法，才能使每次投入大海的都是非教徒。

我们可以把问题简单描述为：总共 n 个人，从 1 开始顺序编号，排成一个圆形队列。从 1 号开始报数，报数到 k 的人出圈。然后从出圈位置的下一个人继续从 1 报数，到 k 的人出圈。已经出圈的人不再参与报数，继续上面的做法，直到环形队

列中最后只留下 m 个人。

即： n 为初始总人数； k 为出圈的报数； m 为最后留下的人数。

请用队列求此问题。

测试用例：

输入：9 2 1 //总人数 n ； 报数数值 k ； 剩下人数 m

输出：3 //剩下人的编号

输入：6 5 1

输出：1 //剩下人的编号

输入：11 3 2

输出：2 7 //剩下人的编号

4) 对上面约瑟夫环问题(Josephus loop problem)，尝试使用顺序表、链表等结构求解。

4 算法设计与实现描述

（书上给出的基本运算外，其它问题必须先给出算法思想或步骤，再给出算法描述）

```
#ifndef MYSTACK_H
#define MYSTACK_H

enum error_code {
    SUCCESS = 0,
    STACK_OVERFLOW,
    STACK_UNDERFLOW
};

// ===== 顺序栈模板 =====
template <typename T> // 声明这是一个模板，T 是一个类型占位符
struct stack {
    T* arr;           // 动态数组，存储类型为 T 的元素
    int top;           // 栈顶指针（作为数组下标，永远是 int，不需要变）
    int capacity;      // 栈的容量
    // 构造函数：初始化栈
    stack(int size) {
        arr = new T[size]; // 创建存储 T 类型数据的数组
        capacity = size;
        top = -1;
    }
    // 析构函数：释放栈内存
    ~stack() {
        delete[] arr;
    }
};
```

// 防御性编程：禁用拷贝构造和赋值操作

```
stack(const stack&) = delete;
```

```

stack& operator=(const stack&) = delete;
// 成员函数：入栈操作
error_code push(T value) { // 传入 T 类型的数据
    if (top == capacity - 1) {
        return STACK_OVERFLOW;
    }
    arr[++top] = value;
    return SUCCESS;
}
// 成员函数：出栈操作
error_code pop(T &value) { // 传出 T 类型的数据
    if (top == -1) {
        return STACK_UNDERFLOW;
    }
    value = arr[top--];
    return SUCCESS;
}
// 成员函数：获取栈顶元素
error_code peek(T &value) {
    if (top == -1) {
        return STACK_UNDERFLOW;
    }
    value = arr[top];
    return SUCCESS;
}
bool isEmpty() {
    return top == -1;
}
};

// ===== 链栈模板 =====
template <typename T> // 链栈也同样声明为模板
struct stacklink {
    struct node {
        T data;          // 节点里存的是 T 类型的数据
        node* next;
    };
    node* top; // 栈顶指针
};

```

```

stacklink() {
    top = nullptr;
}
~stacklink() {
    node* current = top;
    while (current != nullptr) {

```

```

        node* nextNode = current->next;
        delete current;
        current = nextNode;
    }
}

stacklink(const stacklink&) = delete;
stacklink& operator=(const stacklink&) = delete;
void push(T value) {
    node* newNode = new node();
    newNode->data = value;
    newNode->next = top;
    top = newNode;
}

error_code pop(T &value) {
    if (top == nullptr) {
        return STACK_UNDERFLOW;
    }
    node* temp = top;
    value = temp->data;
    top = top->next;
    delete temp;
    return SUCCESS;
}

error_code peek(T &value) {
    if (top == nullptr) {
        return STACK_UNDERFLOW;
    }
    value = top->data;
    return SUCCESS;
}

bool isEmpty() {
    return top == nullptr;
}
};

#endif // MYSTACK_H

```

```

#ifndef QUEUE_H
#define QUEUE_H
using namespace std;
enum Error_code {
    SUCCESS = 0,
    STACK_OVERFLOW,
    STACK_UNDERFLOW
};

```

```

struct clcqueue
{
    int* arr;          // 动态数组，存储队列元素
    int front;         // 队头指针
    int rear;          // 队尾指针
    int capacity;      // 队列容量
    clcqueue(int size) {
        arr = new int[size];
        capacity = size;
        front = rear = 0;
    }
    ~clcqueue() {
        delete[] arr;
    }
    clcqueue(const clcqueue&) = delete;
    clcqueue& operator=(const clcqueue&) = delete;
    bool isFull() {
        return (rear + 1) % capacity == front;
    }
    bool isEmpty() {
        return front == rear;
    }
    Error_code enqueue(int value) {
        if (isFull()) {
            return STACK_OVERFLOW;
        }
        arr[rear] = value;
        rear = (rear + 1) % capacity;
        return SUCCESS;
    }
    Error_code dequeue() {
        if (isEmpty()) {
            return STACK_UNDERFLOW;
        }
        front = (front + 1) % capacity;
        return SUCCESS;
    }
};

struct node
{
    int data; // 数据域
    node* next; // 指针域
    node(int value) : data(value), next(nullptr) {} // 构造函数
};

```

```

struct linkqueue
{
    node* front; // 队头指针
    node* rear;  // 队尾指针
    linkqueue() : front(nullptr), rear(nullptr) {} // 构造函数
    ~linkqueue() { // 析构函数，释放所有节点内存
        while (front != nullptr) {
            node* temp = front;
            front = front->next;
            delete temp;
        }
    }
    linkqueue(const linkqueue&) = delete;
    linkqueue& operator=(const linkqueue&) = delete;
    bool isEmpty() {
        return front == nullptr;
    }
    void enqueue(int value) {
        node* newNode = new node(value);
        if (isEmpty()) {
            front = rear = newNode; // 队列为空时，front 和 rear 都指向新节点
        } else {
            rear->next = newNode; // 将新节点链接到队尾
            rear = newNode; // 更新 rear 指针
        }
    }
    void dequeue() {
        if (isEmpty()) {
            return; // 队列为空，无需出队
        }
        node* temp = front; // 保存当前队头节点
        front = front->next; // 更新 front 指针指向下一个节点
        delete temp; // 释放原队头节点的内存
        if (front == nullptr) { // 如果出队后队列变空，更新 rear 指针
            rear = nullptr;
        }
    }
    int count() {
        int count = 0;
        node* current = front;
        while (current != rear) {
            count++;
            current = current->next;
        }
    }
}

```

```

    }
    return count;
}
};

struct queue
{
    int* arr;          // 动态数组，存储队列元素
    int front;         // 队头指针
    int rear;          // 队尾指针
    int capacity;      // 队列容量
    queue(int size) {
        arr = new int[size];
        capacity = size;
        front = rear = 0;
    }
    ~queue() {
        delete[] arr;
    }
    queue(const queue&) = delete;
    queue& operator=(const queue&) = delete;
    bool isFull() {
        return rear == capacity;
    }
    bool isEmpty() {
        return front == rear;
    }
    void enqueue(int value) {
        if (isFull()) {
            return; // 队列已满，无法入队
        }
        arr[rear] = value;
        rear++;
    }
    void dequeue() {
        if (isEmpty()) {
            return; // 队列为空，无需出队
        }
        // 采用最标准、最高效的顺序队列出队法：只移动头指针
        front++;
    }
    int count() {
        return rear - front; // 返回队列中元素的数量
    }
};

```

```
#endif // QUEUE_H
```

在正式实现算法之前，程序首先通过独立的头文件封装了多种底层线性表结构。其中包括基于动态数组的顺序栈（stack）和基于节点指针的链栈（stacklink），以及顺序循环队列（ciclequeue）、链式队列（linkqueue）和基础顺序队列（queue）。在这些结构的封装中，通过构造函数严谨地初始化了端点指针，并利用析构函数在实例销毁时回收动态内存或释放节点链条，从根源上杜绝了内存泄漏隐患。同时，通过屏蔽拷贝构造与赋值操作，实施了基础的防御性编程。这些健壮的底层结构为后续的表达式解析与状态回溯提供了安全的物理存储依托。

（1）判断读入的字符串表达式中的各括号是否匹配

【算法思想】

借助 stack 后进先出的特性处理括号嵌套。遍历输入的字符串，遇到左侧括号直接调用 push 压入 stack 中保存待用。遇到右侧括号时，调用 peek 取出 stack 顶部的括号进行比对：若 stack 为空或括号类型不一致，则判定为不匹配并提前结束；若类型匹配，则调用 pop 弹出栈顶元素并继续验证后续字符。字符串遍历结束后，若 stack 完全为空则判定匹配，否则说明左括号有剩余，判定不匹配。

【算法步骤】

- ① 状态初始化：创建一个字符类型的 stack 用于暂存左侧括号状态。
- ② 遍历与入栈：循环读取输入字符串中的每一个字符。若为左括号，则直接调用 push 入栈。
- ③ 校验与出栈：若为右括号，先调用 peek 获取栈顶元素。若获取失败则提示无左括号并返回；若类型匹配则调用 pop 消除该对括号；若类型冲突则提示不匹配并返回。
- ④ 终局清算：遍历完毕后，再次调用 peek 检查。若 stack 仍有元素残留，说明多出左括号；若为空，则输出括号匹配成功。

【算法描述和实现】

```
void deside(stack<char> &s, string input) {
    char topChar;
    for (char c : input) {
        if (c == '(' || c == '{' || c == '[') {
            s.push(c);
        } else {
            if (s.peek(topChar) != SUCCESS) {
                cout << "无左括号";
                return;
            }
            if ((c == ')' && topChar == '(') ||
                (c == '}' && topChar == '{') ||
                (c == ']' && topChar == '[')) {
                s.pop(topChar);
            } else {
                cout << "括号不匹配";
                return;
            }
        }
    }
}
```



```

    if (s.peek(topChar) == SUCCESS) {
        cout << "无右括号";
    } else {
        cout << "括号匹配";
    }
}
}

```

(2) 求出所有可能的出栈序列

【算法思想】

结合 stack 操作与回溯算法生成所有可能的序列。递归过程中始终面临两条分支选择。第一条是入栈分支，若当前准备入栈的数字未超过总数设定值，则将其 push 入 stack 中并向下探索，探索返回后调用 pop 弹出以恢复现场。第二条是出栈分支，若 stack 不为空，则 pop 出栈顶元素存入路径记录中并向下探索，探索返回后将其移除并重新 push 回 stack 以撤销操作。当路径中收集的数字数量达到总数设定值时，即找到一条完整合法序列。

【算法步骤】

- ①终点判定：检查路径记录容器 path 的长度是否等于目标总数 n。若相等，则全局总数计数器累加一，并结束当前递归分支。
- ②入栈探测：若当前数字小于等于总数 n，调用 push 压入容器 s，随后对下一个递增数值发起递归。递归结束后，调用 pop 恢复执行前的现场环境。
- ③出栈探测：若容器 s 内部存在元素，调用 pop 提取顶部数值并追加到 path 容器中，随后发起递归。递归结束后，将 path 末尾回退，并调用 push 将数值的原主。
- ④启动穷举：在主函数中初始化容器与目标数值，发起首次调用，最终输出统计完毕的合法路线总数。

【算法描述和实现】

```

#include <iostream>
#include "mystack.h"
#include <vector>
using namespace std;
int totalcount = 0;

void generateSequences(int n, int current_num, stack<int>& s, vector<int>& path) {
    if (path.size() == n)
    {
        totalcount++;
        return;
    }
    if (current_num <= n)
    {
        s.push(current_num);
        generateSequences(n, current_num + 1, s, path);
        int temp;
        s.pop(temp);
    }
    if (!s.isEmpty())
    {

```

```

        int x;

        s.pop(x);

        path.push_back(x);

        generateSequences(n, current_num, s, path);

        path.pop_back();

        s.push(x);

    }
}

int main() {

    stack<int> s(100);

    vector<int> path;

    int n ;

    cin >> n;

    cout<<"输入 n 的值为: "<<n<<endl;

    generateSequences(n, 1, s, path);

    cout << "总的出栈序列数: " << totalcount << endl;

}

```

(3) 用队列求解约瑟夫环问题

【算法思想】

利用 queue 先进先出的机制模拟环形报数过程。当 queue 内部的总人数严格大于要求留下的目标人数时，持续执行淘汰逻辑。在每轮报数中，将排在前面的若干人员（步数减一个）依次调用 dequeue 出队，并立刻调用 enqueue 重新排到队尾，以此模拟人员环形轮转；随后，对刚好排在队头的淘汰目标人员直接调用 dequeue 废弃。不断重复该轮转淘汰过程，直到内部剩余人数达标，最后依次出队并输出留下的人员编号即可。

【算法步骤】

- ① 循环判定：启动外部循环，当 queue 内部有效元素的计算总数大于指定留存人数 m 时，维持淘汰运转。
- ② 轮转模拟：利用内部循环，将其余人员（即前 k 减一个人员）提取数值后调用 dequeue 出队，并调用 enqueue 塞入队尾，营造环形越过效果。
- ③ 处决执行：内层循环结束后，直接对队首的目标人员调用 dequeue 进行删除淘汰。
- ④ 安全提取：淘汰循环彻底结束后，借助 isEmpty 判断不断提取并输出队首人员坐标，完成幸存者名单播报。

【算法描述和实现】

```

void live(cyclequeue &q,int k,int m)
{
    while ((q.rear+q.capacity-q.front)%q.capacity>m)
    {
        for (int i = 1; i < k; i++)
        {
            int temp=q.arr[q.front]; // 获取队头元素
            q.dequeue(); // 将队头元素出队并重新入队，实现循环移动
            q.enqueue(temp);
        }
        q.dequeue(); // 删除第 k 个元素
    }
}

```

```

    }

    cout<<"剩余元素为: ";
    while (!q.isEmpty())
    {
        int temp=q.arr[q.front]; // 获取队头元素
        cout<<temp<<" "; // 输出队头元素
        q.dequeue(); // 将队头元素出队
    }

    cout<<endl;
}
}

```

(4) 使用链队列与顺序队列重现约瑟夫环求解

【算法思想】

分别使用底层的链式队列（linkqueue）与顺序队列（queue）重构上述业务逻辑，以验证在高层算法视角下，只要底层接口封装一致，物理存储形式的改变不会干扰上层轮转机制的运转。

【算法步骤】

- ① 链式重构：利用 linkqueue 的 count 接口测算元素余量维持外层循环。通过存取节点数据结合出入队指令实现模拟轮转。
- ② 顺序重构：利用常规 queue 同样执行内部元素的出入队更替，实现环形报数的推进。
- ③ 校验输出：针对两种不同物理结构，分别清空并输出存活编号，对比运行结果的一致性。

【算法描述和实现】

```

void live1(linkqueue &q,int k,int m)
{
    while (q.count()+1>m)
    {
        for (int i = 1; i < k; i++)
        {
            int temp=q.front->data; // 获取队头元素
            q.dequeue(); // 将队头元素出队并重新入队，实现循环移动
            q.enqueue(temp);
        }
        q.dequeue(); // 删除第 k 个元素
    }

    cout<<"剩余元素为: ";
    while (!q.isEmpty())
    {
        int temp=q.front->data; // 获取队头元素
        cout<<temp<<" "; // 输出队头元素
        q.dequeue(); // 将队头元素出队
    }

    cout<<endl;
}

void live2(queue &q, int k, int m)
{

```

```

while (q.count()>m)
{
    for (int i = 1; i < k; i++)
    {
        int temp=q.arr[q.front]; // 获取队头元素
        q.dequeue(); // 将队头元素出队并重新入队，实现循环移动
        q.enqueue(temp);
    }
    q.dequeue(); // 删除第 k 个元素
}

cout<<"剩余元素为: ";
while (!q.isEmpty())
{
    int temp=q.arr[q.front]; // 获取队头元素
    cout<<temp<<" "; // 输出队头元素
    q.dequeue(); // 将队头元素出队
}

cout<<endl;
}

```

5 运行结果截图及说明

(1)

<pre> int main() { stack<char> s(100), s2(100); string input1 = "{[(){} }"; cout <<"第一次测试: " << endl; cout << "输入的括号字符串: " << input1 << endl; deside(s, input1); string input2 = "{[()}]"; cout << "\n\n第二次测试: " << endl; cout << "输入的括号字符串: " << input2 << endl; deside(s2, input2); return 0; } </pre>	第一次测试: 输入的括号字符串: {[(){} } 括号匹配 第二次测试: 输入的括号字符串: {[()}] 括号不匹配
---	---

说明：程序针对数学表达式中的各种括号进行了嵌套验证。针对正确嵌套的输入实例，程序准确输出了匹配成功的结论；面对因层级错乱导致的不匹配情况，程序也能即时中断并准确反馈校验失败的信息

(2)

<pre> int main() { stack<int> s(100); vector<int> path; int n ; cin >> n; cout<<"输入n的值为:"<<n<<endl; generateSequences(n, 1, s, path); cout << "总的出栈序列数: " << totalcount << endl; } </pre>	5 输入n的值为: 5 总的出栈序列数: 42
---	--

说明:对指定的数字输入序列进行深度回溯运算。在设置输入总数为五时，程序经过穷举试探与路径复原，准确输出了四十二种合法的出栈序列总数。

(3)

```
int main() {
    int n;
    cout << "请输入队列的容量: ";
    cin >> n;
    cllcqueue q(n+1); // 创建一个容量为 n+1 的循环队列
    for (int i = 1; i <= n; ++i) {
        q.enqueue(i); // 将 1 到 n 的元素入队
    }
    int k;
    cout << "请输入要删除的第 k 个元素: ";
    cin >> k;
    cout << "要留下m个元素: ";
    int m;
    cin >> m;
    live(q, k, m); // 调用 live 函数进行处理
    return 0;
}
```

请输入队列的容量: 9
请输入要删除的第 k 个元素: 2
要留下m个元素:1
剩余元素为: 3

请输入队列的容量: 6
请输入要删除的第 k 个元素: 5
要留下m个元素:1
剩余元素为: 1

请输入队列的容量: 11
请输入要删除的第 k 个元素: 3
要留下m个元素:2
剩余元素为: 2 7

说明: 展示了使用链式队列处理约瑟夫环问题的三组核心测试用例。无论是指定留存单人还是多人, 程序均能通过出队入队的循环流转过滤掉淘汰者。输出的安全编号坐标与标准题解完全一致, 验证了淘汰逻辑的准确性。

(4)

请输入链队列的容量: 11
请输入要删除的第 k 个元素: 3
要留下m个元素:2
剩余元素为: 2 7

请输入顺序队列的容量: 11
请输入要删除的第 k 个元素: 3
要留下m个元素:2
剩余元素为: 2 7

说明: 将底层结构更换为顺序队列后, 使用相同的测试数据再次运行系统。输出结果与链式队列处理方案相同。

6 总结、心得和建议

本次综合实验的核心任务是围绕 `stack` 与 `queue` 这两种受限线性表展开深度应用开发。它们在物理层面依然依托于底层的顺序结构或链式结构, 但通过在接口端施加严格的访问方位限制, 为特定场景下的状态管理与任务调度提供了极其高效的安全范式。

【心得】

(1) 栈结构的嵌套依赖管理: 括号匹配问题充分揭示了 `stack` 结构的后进先出特性在处理层级逻辑时的绝对优势。程序将左侧括号作为待匹配的上下文状态推入其中。遇到右侧括号时, 直接提取顶部元素进行核对销毁。

(2) 系统调用栈与回溯的同构性: 出栈序列穷举任务巧妙利用了递归。为了模拟所有进出路径, 系统在递归分支点面临 `push` 与 `pop` 的状态抉择。递归深入探索后, 必须严谨地撤销当前状态, 恢复上一层级的现场环境, 防止遗留脏数据影响后续探索路径。

(3) 队列轮转机制对复杂模型的降维: 约瑟夫环问题的求解全面展示了 `queue` 先进先出特性的实战效能。传统的数组索引模拟法需要手动且频繁地处理越界重置与逻辑跳过。通过引入 `queue` 机制, 程序将复杂的环形计算抽象为极简的 `enqueue` 与 `dequeue` 循环。未被淘汰的人员从前端离开立刻接入尾部, 触发淘汰规则的人员则直接出队销毁, 大幅降低了工程维护难度。

【建议】

(1) 引入针对回溯算法的压力测试机制：目前实验采用的序列总数较小，程序的递归表现平稳。但在面对庞大输入规模时，指数级增长的递归深度极易引发系统调用栈溢出或内存耗尽。建议在后续类似实验中设定硬性的性能压测瓶颈，促使学生主动探索非递归迭代等替代架构。

(2) 深化底层物理结构适用边界的对比：本次实验虽然验证了循环队列与链式队列在宏观结果上的一致性，但忽略了二者在极限吞吐量下的内存分配差异。建议后续引导学生针对不同业务场景的访问频次，进行底层容量扩充成本与指针寻址开销的专项对比研究，以培养更为严谨克制的工程思维 。